

A PROJECT REPORT ON

PLAGIARISM AND AI-GENERATED CONTENT DETECTOR FOR ACADEMIC REPORTS

Domain: NLP / Education

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 Software Specification	
V	SYSTEM ANALYSIS	10
	5.1 Feasibility Study	

	5.2 About the Software	
VI	SYSTEM DESIGN	13
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	21
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	28
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	29

CHAPTER I

INTRODUCTION

Academic integrity is fundamental to education. Assignments, project reports, theses and research papers are meant to reflect a student's own understanding and effort. However, the easy availability of online content has made copy-paste plagiarism common, and students increasingly use paraphrasing tools to disguise copied text. Since 2022, large language model chatbots can generate entire essays and reports in seconds.

Faculty members evaluating dozens of reports cannot manually search the internet or compare each report with previous years' submissions. Simple string matching misses paraphrased plagiarism, and commercial tools are expensive for many colleges. Faculty also need explainable reports that show exactly which passages match which sources, rather than a single percentage.

This project, IntegrityCheck, compares a submitted report with a local corpus of previous submissions and reference documents and with web sources, detecting exact copying through fingerprinting and paraphrased copying through sentence embeddings. It also estimates whether sections appear machine-generated using language model perplexity and burstiness features. Matches are highlighted with sources and percentages in a PDF similarity report for faculty. The AI-text estimate is presented as an indicator for further discussion, not as proof.

LIMITATIONS

1. Copy-paste work is hard to check manually.
2. Paraphrased plagiarism escapes simple matching.
3. Faculty need quick, explainable reports.
4. AI-generated text is a new challenge not handled by older tools.

1.1 History of Domain

Plagiarism detection software began with tools for source code such as MOSS (1994) at Stanford, which used document fingerprinting with winnowing. Turnitin,

launched in 1998, compared student papers with a large database of web pages and submissions and became widely used in universities. In India, the UGC introduced regulations on academic integrity in 2018 and promotes the use of similarity detection tools such as the URKUND/Ouriginal (DrillBit) service.

Research moved from exact matching to paraphrase detection using semantic similarity and, more recently, to detection of machine-generated text. Methods such as GLTR (2019) and DetectGPT (2023) use statistical properties of language model probabilities, while watermarking approaches have also been proposed. These detectors have known limitations, so results must be interpreted carefully.

1.2 Overview of the Project

The faculty or student uploads a report (PDF or DOCX). Text is extracted, cleaned, and split into sentences and overlapping chunks. For exact and near-exact copying, word 5-gram shingles are hashed and winnowed into fingerprints, which are looked up in an Elasticsearch index of the local corpus. For paraphrase detection, each sentence is embedded with a Sentence-Transformer, and approximate nearest-neighbour search finds semantically similar sentences in the corpus; selected key sentences are also searched on the web through a search API.

For the AI-generated content estimate, each section is scored using a small language model (GPT-2) to compute perplexity and burstiness (variation of perplexity across sentences), along with a classifier trained on human-written and AI-generated academic text. The system produces an overall similarity percentage, a source-wise breakdown, highlighted passages and an AI-likelihood indicator per section, and generates a downloadable PDF report.

1.3 Motivation

Fair evaluation depends on knowing which work is genuinely the student's own, and faculty need tools that are affordable and explainable.

- **Motivation:** To provide colleges with an explainable, affordable integrity checker covering copying, paraphrasing and AI-generated text.
- **Outcome:** Improved academic honesty, faster evaluation, and constructive feedback to students about citation practices.

1.4 Aim of the Project

The project is entitled as “**PLAGIARISM AND AI-GENERATED CONTENT DETECTOR FOR ACADEMIC REPORTS**”. The aim of this project is to develop a tool that detects copied and paraphrased content in academic reports by comparing them with a document corpus and web sources, estimates sections that may be AI-generated, and produces an explainable similarity report for faculty.

1.5 Objectives

- To extract and preprocess text from PDF and DOCX reports.
- To detect exact matches using shingling, hashing and winnowing.
- To detect paraphrases using sentence embeddings.
- To estimate AI-generated text using perplexity and burstiness features.
- To generate highlighted PDF similarity reports.

1.6 Literature Review

Broder (1997): Introduced the concept of shingling and resemblance of documents, using sets of contiguous word sequences to estimate similarity between documents efficiently.

Schleimer, Wilkerson and Aiken (2003): Proposed winnowing, a local algorithm for document fingerprinting that guarantees detection of matches longer than a threshold, used in the MOSS plagiarism detection system.

Reimers and Gurevych (2019): Sentence-BERT produces semantically meaningful sentence embeddings, enabling efficient paraphrase mining by cosine similarity.

Gehrmann, Strobel and Rush (2019): Presented GLTR, a tool that detects generated text by visualising how likely each word is under a language model, improving human

detection rates.

Mitchell et al. (2023): Proposed DetectGPT, a zero-shot method that detects machine-generated text using the curvature of the model's log probability function.

Liang et al. (2023): Showed that GPT detectors can be biased against non-native English writers, highlighting that AI detection results should not be used as sole evidence.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

In the existing system, faculty manually read reports and use web searches for suspicious sentences. Some colleges use commercial similarity tools with limited licences. Comparison with previous years' local project reports is rarely done, and there is no support for AI-generated text detection.

DISADVANTAGES

- Manual checking is slow and incomplete.
- Paraphrased content is missed.
- Local previous-year reports are not compared.
- Commercial tools are costly.
- No AI-text indicator.

2.2 Proposed System

The proposed IntegrityCheck system combines fingerprint-based exact matching, embedding-based paraphrase detection, web source search and an AI-text estimator in one tool. It maintains a local corpus of submissions and references and generates explainable, highlighted reports for faculty.

ADVANTAGES

- Detects both copied and paraphrased content.
- Compares with local submissions and web sources.
- Explainable highlighted report with sources.
- AI-text indicator with appropriate caution.
- Low cost using open-source components.

2.3 Project Schedule

The development of IntegrityCheck follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the IntegrityCheck system are listed below.

- **Upload:** Accept PDF and DOCX documents.
- **Text extraction:** Extract text and exclude references and quotations optionally.
- **Chunking:** Split into sentences and chunks.
- **Fingerprinting:** Generate shingles and winnowed hashes.
- **Exact matching:** Find matching fingerprints in the corpus.
- **Paraphrase detection:** Find semantically similar sentences.
- **AI-text estimation:** Compute perplexity/burstiness and classifier score.
- **Report generation:** Highlighted PDF report with sources and percentages.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Performance:** Analyse a 40-page report within 2 minutes.
- **Accuracy:** Paraphrase detection F1 above 0.85 on test pairs.
- **Explainability:** Every match shows the source passage.

- **Privacy:** Submissions stored only in the institution's corpus with permission.
- **Scalability:** Corpus of 50,000 documents.
- **Fairness:** AI-text estimate shown with confidence and disclaimer.

3.3 User Requirements

- Upload a report and get results quickly
- See which sentences match which sources
- Exclude quotes and references
- Download a PDF report
- Add reference documents to the corpus

3.4 System Requirements

- Flask API with Python NLP libraries
- Elasticsearch for fingerprint and vector search
- React front end
- Web search API (optional)
- GPU optional for faster embedding

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System : Intel i5 Processor 2.4 GHz or above
Hard Disk : 512 GB SSD
RAM : 16 GB
GPU : Optional NVIDIA GPU
Monitor : 15" colour monitor

4.2 Software Specification

Operating System : Ubuntu 22.04 / Windows 10
Coding Language : Python 3.10, JavaScript
NLP : Sentence-Transformers, HuggingFace (GPT-2), NLTK
Search : Elasticsearch 8 (text + dense vector)
Back End : Flask
Front End : React JS
Report : ReportLab (PDF)

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed IntegrityCheck system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

All algorithms (shingling, winnowing, SBERT, GPT-2 perplexity) are implementable with open-source libraries, and Elasticsearch supports both text and vector search. The project is technically feasible.

ECONOMIC FEASIBILITY

The software is free and runs on a single college server, avoiding per-document licence costs. It is economically feasible.

OPERATIONAL FEASIBILITY

Faculty upload a file and read a report; the workflow mirrors existing commercial tools. It is operationally feasible.

5.2 About the Software

PYTHON

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an ecosystem of third-party packages available through the Python Package Index (PyPI),

which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain
- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

SENTENCE-TRANSFORMERS (SBERT)

Sentence-Transformers is a Python framework for state-of-the-art sentence, paragraph and document embeddings. It modifies BERT using siamese and triplet network structures so that semantically similar sentences are placed close together in vector space.

Similarity between two texts is calculated using cosine similarity of their embeddings, which captures meaning rather than exact words; for example 'ML engineer' and 'machine learning developer' get a high score.

HUGGINGFACE TRANSFORMERS

HuggingFace Transformers is an open-source library that provides thousands of pre-trained transformer models for tasks such as text classification, named entity recognition, question answering and translation, in more than 100 languages.

Multilingual models such as mBERT, XLM-RoBERTa and IndicBERT understand Tamil and English, so a single model can be fine-tuned to classify text in both languages.

- Pre-trained multilingual models
- Simple pipeline() API
- Works with both PyTorch and TensorFlow

ELASTICSEARCH

Elasticsearch is a distributed, RESTful search and analytics engine built on Apache Lucene. It stores documents as JSON and builds inverted indexes for very fast full-text

search.

It also supports dense-vector fields and k-nearest-neighbour search, so both exact text matches and semantic embedding matches can be found from the same index.

FLASK

Flask is a lightweight WSGI web application framework written in Python. It is classified as a micro-framework because it does not require particular tools or libraries; it has no database abstraction layer or form validation built in, but extensions can add these features as needed.

Flask provides URL routing, request handling, Jinja2 templating and a development server. Because it is small and simple, it is ideal for exposing a trained machine learning model as a REST API or building a small web application quickly.

- Minimal and easy to learn
- Jinja2 templates for HTML pages
- Large set of extensions (Flask-Login, Flask-SQLAlchemy, Flask-CORS)

REACT JS

React is an open-source JavaScript library for building user interfaces, maintained by Meta and a community of developers. It lets developers build reusable UI components and manages the view layer of web applications efficiently using a virtual DOM.

React applications are composed of components that hold their own state and re-render automatically when data changes. With libraries such as React Router and Axios, a single-page application can communicate with REST APIs and give the user a fast, app-like experience.

- Component based, reusable UI
- Virtual DOM for fast rendering
- Large ecosystem of UI libraries

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of IntegrityCheck is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

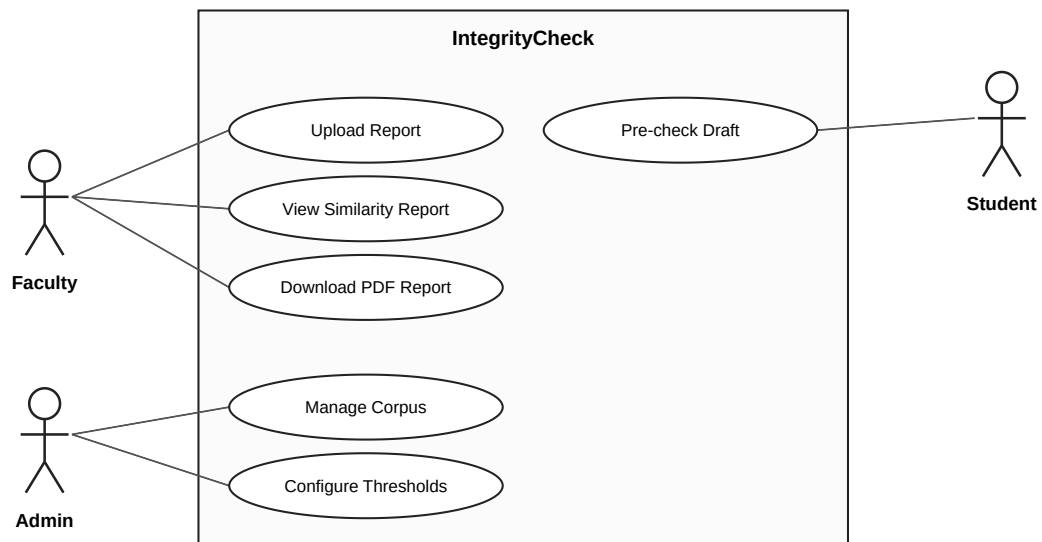


Fig 1: Use case diagram of IntegrityCheck

6.2 System Architecture

IntegrityCheck has a processing pipeline that converts documents into fingerprints and embeddings, a search layer based on Elasticsearch, an AI-text analysis module, and a report generator.

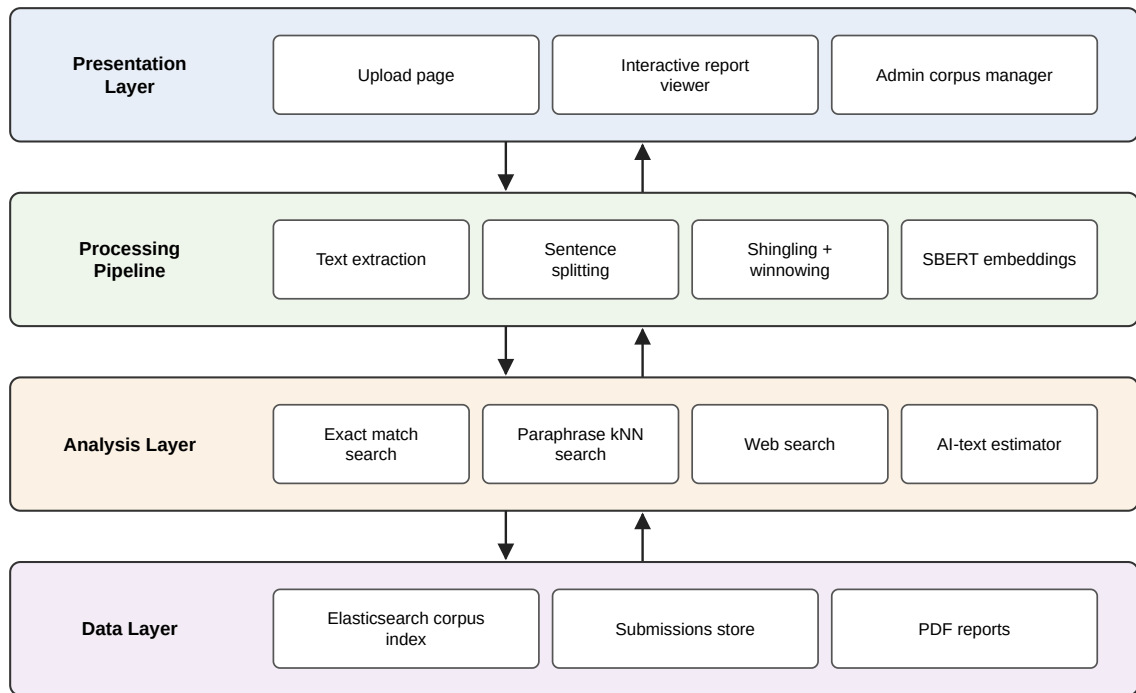


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

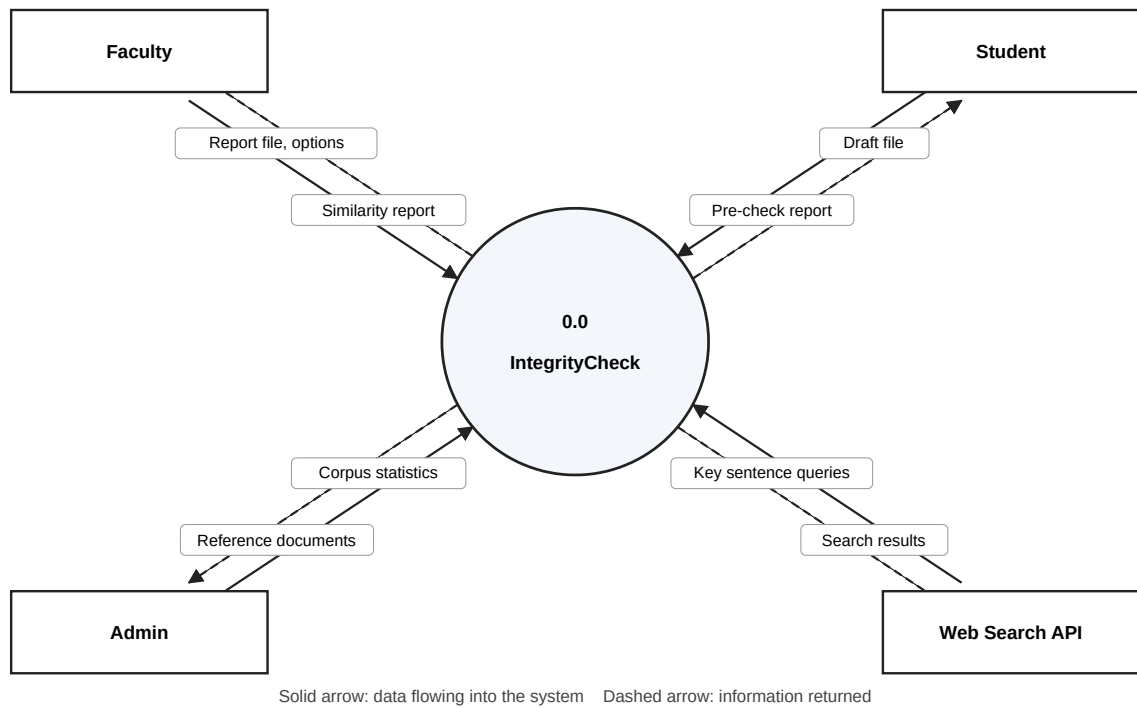


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

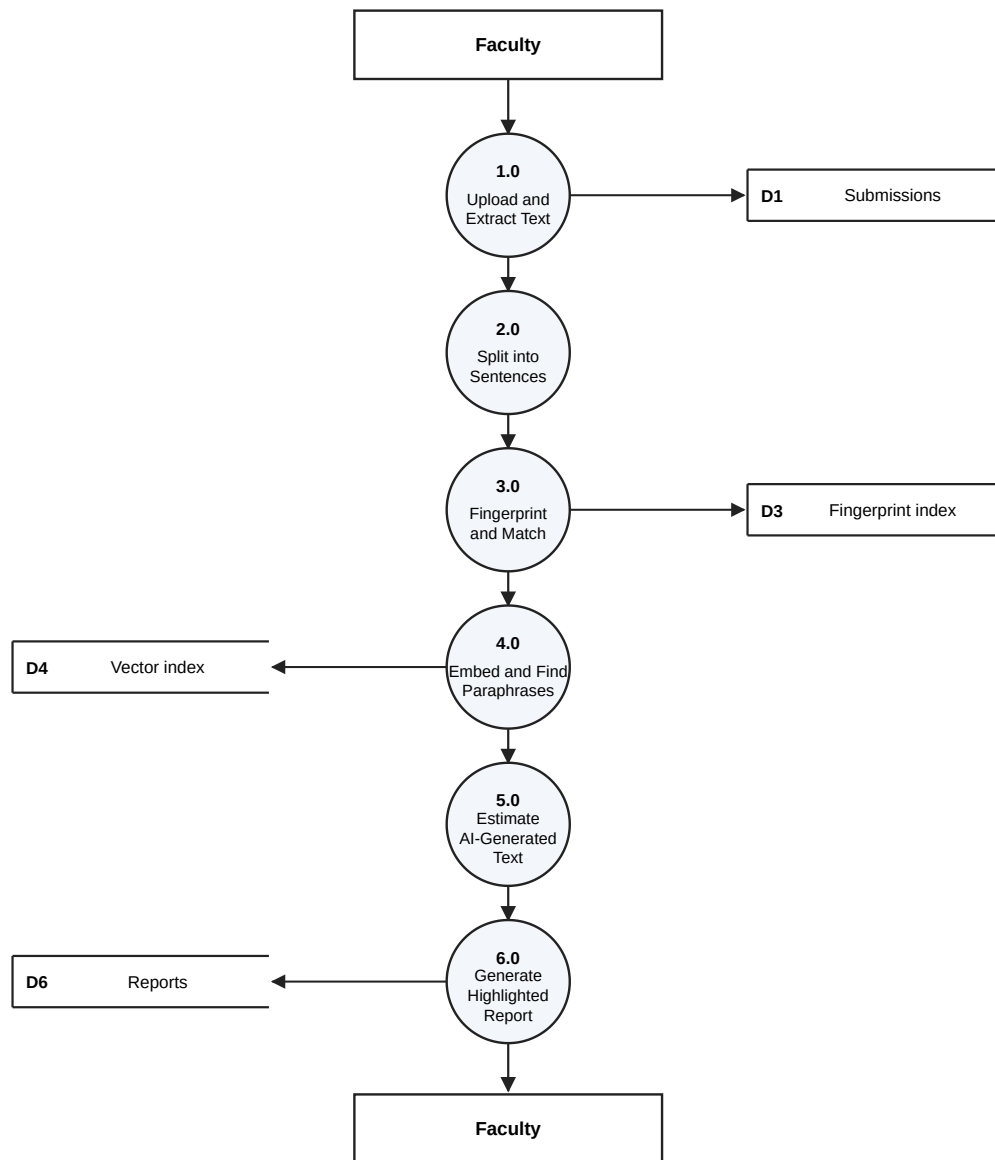


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

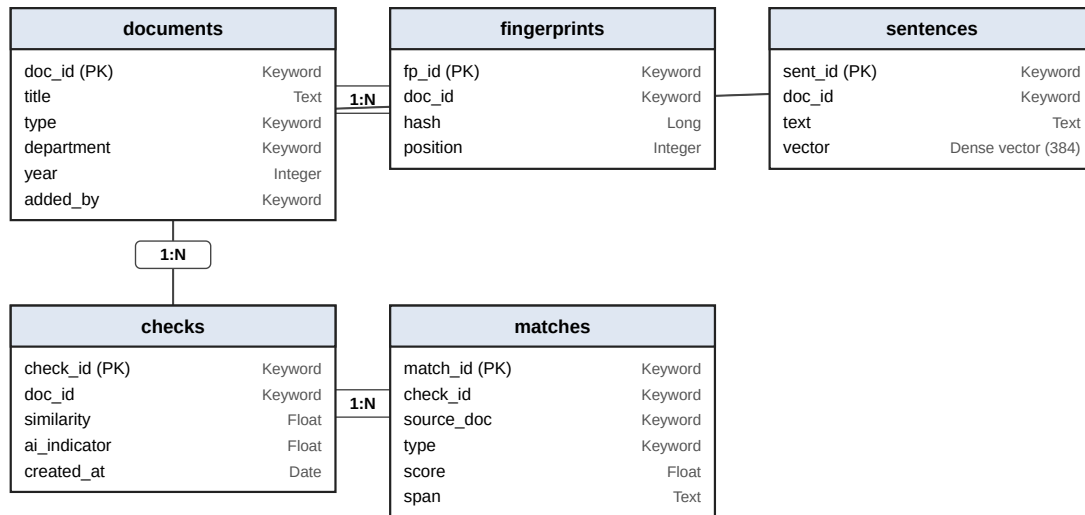


Fig 5: Entity relationship diagram

6.5 Table Design

DOCUMENTS

FIELD NAME	DATA TYPE	DESCRIPTION
doc_id	Keyword	Primary key
title	Text	Document title
type	Keyword	submission / reference / paper
department	Keyword	Department
year	Integer	Year
added_by	Keyword	Uploader

FINGERPRINTS

FIELD NAME	DATA TYPE	DESCRIPTION
fp_id	Keyword	Primary key
doc_id	Keyword	Foreign key to documents
hash	Long	Winnowed shingle hash
position	Integer	Word offset

SENTENCES

FIELD NAME	DATA TYPE	DESCRIPTION
------------	-----------	-------------

sent_id	Keyword	Primary key
doc_id	Keyword	Foreign key to documents
text	Text	Sentence
vector	Dense vector (384)	SBERT embedding

CHECKS

FIELD NAME	DATA TYPE	DESCRIPTION
check_id	Keyword	Primary key
doc_id	Keyword	Checked document
similarity	Float	Overall %
ai_indicator	Float	AI-likelihood
created_at	Date	Time

MATCHES

FIELD NAME	DATA TYPE	DESCRIPTION
match_id	Keyword	Primary key
check_id	Keyword	Foreign key to checks
source_doc	Keyword	Source document / URL
type	Keyword	exact / paraphrase
score	Float	Similarity score
span	Text	Matched text

6.6 Algorithm Design

The detection pipeline combines three complementary techniques.

1. **Upload document and extract text** – pdfplumber / python-docx; quoted text and bibliography optionally removed.
2. **Split into sentences and chunks** – NLTK sentence tokenizer; chunks of 5 sentences with overlap for the AI estimator.
3. **Generate fingerprints (shingling + hashing)** – Words are normalised and 5-gram shingles hashed; winnowing with window 4 selects fingerprints.

4. **Compare with stored corpus for exact matches** – Fingerprints are looked up in Elasticsearch; consecutive matching positions form matched passages.
5. **Use sentence embeddings for paraphrase detection** – SBERT (all-MiniLM-L6-v2) vectors; kNN search; cosine ≥ 0.85 and no exact match = paraphrase.
6. **Compute perplexity/burstiness features for AI-text estimate** – GPT-2 perplexity per sentence; mean and standard deviation per chunk, plus a logistic regression classifier.
7. **Highlight sources and percentages** – Similarity % = matched words / total words; per-source contribution computed.
8. **Generate PDF similarity report** – ReportLab produces a report with colour-coded highlights and a source list.

PSEUDO CODE

```
CHECK(doc):
  text <- CLEAN(EXTRACT(doc)); sents <- SENT_SPLIT(text)
  fps <- WINNOW(HASH(SHINGLES(words(text), 5)), w=4)
  exact <- MERGE_RUNS(ES.lookup(fps))
  FOR s IN sents NOT covered by exact:
    (src, sim) <- ES.knn(SBERT(s), k=3)[0]
    IF sim >= 0.85 THEN paraphrase.add(s, src, sim)
  FOR chunk IN CHUNKS(sents, 5):
    ppl <- [GPT2_PERPLEXITY(s) for s in chunk]
    ai[chunk] <- LOGREG([mean(ppl), std(ppl), len, ttr])
  similarity <- matched_words / total_words
  RETURN REPORT(similarity, exact, paraphrase, ai)
```

6.7 Modules

The modules are grouped by the Faculty, Student and Admin users.

MODULES

FACULTY

1. UPLOAD REPORT
2. VIEW SIMILARITY REPORT
3. DOWNLOAD PDF REPORT

STUDENT

1. PRE-CHECK DRAFT

ADMIN

1. MANAGE CORPUS

2. CONFIGURE THRESHOLDS

FACULTY

UPLOAD REPORT

Faculty upload a student report, choose options (exclude quotes, exclude bibliography, minimum match length) and select whether to add it to the corpus.

VIEW SIMILARITY REPORT

Faculty view the overall similarity, source-wise percentages, highlighted passages and AI-likelihood per section.

DOWNLOAD PDF REPORT

A formatted PDF report can be downloaded and shared with the student.

STUDENT

PRE-CHECK DRAFT

Students can check their draft (without adding it to the corpus) to correct missing citations before final submission.

ADMIN

MANAGE CORPUS

Admin uploads reference books, previous years' reports and papers into the corpus and manages access.

CONFIGURE THRESHOLDS

Admin sets similarity thresholds and AI-indicator settings.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

IntegrityCheck was implemented in Python using Flask for the API. Elasticsearch 8 stores fingerprints as keyword/long fields and sentence embeddings as dense vectors with HNSW indexing for kNN search. GPT-2 (small) from HuggingFace computes perplexity. Reports are generated with ReportLab.

7.2 Implementation

The corpus was initialised with 1,800 previous project reports from the department (with permission) and 600 reference documents. Paraphrase detection was evaluated on the Microsoft Research Paraphrase Corpus and on 400 manually created paraphrase pairs from project reports. The AI-text classifier was trained on 2,000 human-written report paragraphs from before 2022 and 2,000 AI-generated paragraphs on the same topics.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.

- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	Upload PDF	report.pdf (40 pages)	Text extracted and analysed	Pass
TC02	Exact copy	Paragraph copied from corpus report	Exact match with source	Pass
TC03	Paraphrase	Reworded paragraph	Paraphrase match (sim 0.89)	Pass
TC04	Original text	Student's own paragraph	No match	Pass
TC05	Exclude quotes	Quoted sentence with citation	Not counted when option on	Pass
TC06	AI-text indicator	AI-generated introduction	High AI-likelihood shown with caution	Pass
TC07	PDF report	Click Download	PDF with highlights and sources	Pass
TC08	Large corpus speed	Corpus of 2,400 docs	Result in < 2 minutes	Pass

PERFORMANCE EVALUATION

Evaluation results:

Task	Precision	Recall	F1
Exact / near-exact copy detection	0.98	0.96	0.97
Paraphrase detection (SBERT ≥ 0.85)	0.88	0.84	0.86
AI-generated paragraph indicator	0.84	0.79	0.81

7.4 Source Code

fingerprint.py

```
import re, hashlib
from collections import deque
```

```

def normalise(text):
    text = text.lower()
    text = re.sub(r"^[a-z0-9\s]", " ", text)
    return text.split()

def shingles(words, k=5):
    for i in range(len(words) - k + 1):
        yield i, " ".join(words[i:i + k])

def h(s):
    return int(hashlib.md5(s.encode()).hexdigest()[:15], 16)

def winnow(words, k=5, w=4):
    """Schleimer et al. (2003): pick the minimum hash in each window of w
    shingle hashes."""
    hashes = [(h(s), pos) for pos, s in shingles(words, k)]
    fps, window, last = [], deque(), None
    for i, hp in enumerate(hashes):
        window.append(hp)
        if len(window) > w:
            window.popleft()
        if len(window) == w:
            m = min(window, key=lambda x: (x[0], -x[1]))
            if m != last:
                fps.append(m); last = m
    return fps
# list of (hash, word position)

```

detector.py

```

import numpy as np, torch, nltk, joblib
from elasticsearch import Elasticsearch
from sentence_transformers import SentenceTransformer
from transformers import GPT2LMHeadModel, GPT2TokenizerFast
from fingerprint import normalise, winnow

es = Elasticsearch("http://localhost:9200")
sbert = SentenceTransformer("all-MiniLM-L6-v2")
tok = GPT2TokenizerFast.from_pretrained("gpt2"); lm =
GPT2LMHeadModel.from_pretrained("gpt2").eval()
ai_clf = joblib.load("models/ai_text_logreg.pkl")

def exact_matches(words):
    fps = winnow(words)
    res = es.search(index="fingerprints", size=10000,
                    query={"terms": {"hash": [h for h, _ in fps]}})
    by_hash = {}
    for hit in res["hits"]["hits"]:
        by_hash.setdefault(hit["_source"]["hash"], set()).add(hit["_source"]
["doc_id"])
    covered = {}
    for hsh, pos in fps:
        for doc in by_hash.get(hsh, ()):
            for p in range(pos, pos + 5):
                covered.setdefault(p, doc)
    return covered
# word position -> source
doc

def paraphrases(sentences, threshold=0.85):
    vecs = sbert.encode(sentences, normalize_embeddings=True)
    out = []

```

```

    for s, v in zip(sentences, vecs):
        r = es.search(index="sentences", knn={"field": "vector", "query_vector":
v.tolist(), "k": 3, "num_candidates": 50})
        best = r["hits"]["hits"][0] if r["hits"]["hits"] else None
        if best and best["_score"] * 2 - 1 >= threshold:          # ES cosine
score = (1 + cos) / 2
            out.append({"sentence": s, "source": best["_source"]["doc_id"],
                        "source_text": best["_source"]["text"], "score":
round(best["_score"] * 2 - 1, 3)})
    return out

def perplexity(sentence):
    enc = tok(sentence, return_tensors="pt", truncation=True, max_length=256)
    with torch.no_grad():
        loss = lm(**enc, labels=enc["input_ids"]).loss
    return float(torch.exp(loss))

def ai_indicator(sentences):
    scores = []
    for i in range(0, len(sentences), 5):
        chunk = sentences[i:i + 5]
        ppl = [perplexity(s) for s in chunk if len(s.split()) > 5]
        if len(ppl) < 3: continue
        words = " ".join(chunk).split()
        feats = [[np.mean(ppl), np.std(ppl), np.mean([len(s.split()) for s in
chunk]), len(set(words)) / len(words)]]
        scores.append((i, float(ai_clf.predict_proba(feats)[0, 1])))
    return scores

def check(text):
    words = normalise(text)
    covered = exact_matches(words)
    sents = [s for s in nltk.sent_tokenize(text) if len(s.split()) > 6]
    para = paraphrases(sents)
    para_words = sum(len(p["sentence"].split()) for p in para)
    similarity = min(100, round(100 * (len(covered) + para_words) /
max(len(words), 1), 1))
    sources = {}
    for doc in covered.values():
        sources[doc] = sources.get(doc, 0) + 1
    return {"similarity": similarity, "exact_sources": sources, "paraphrases":
para, "ai_chunks": ai_indicator(sents)}

```

7.5 Screen Layout

SCREENSHOTS:

UPLOAD FOR CHECKING

https://integritycheck.app/

IntegrityCheck

UploadReportCorpusSubmissionsSettings

Check a report

Report (PDF / DOCX)

21UCS1045_Project_Report.pdf (2.1 MB, 46 pages)

Exclude

Quotes, Bibliography

Minimum match length

8 words

Add to institutional corpus

Yes (final submission)

Start Check

What is checked

Exact and near-exact copying (fingerprints).

Paraphrased sentences (meaning-based).

Web sources for key sentences.

AI-generated text indicator (advisory only).

SIMILARITY REPORT

https://integritycheck.app/

IntegrityCheck

UploadReportCorpusSubmissionsSettings

Similarity report - 21UCS1045 Project Report

Overall similarity 18% | AI indicator: Medium in Ch. 1

Exact matches	11% (4 sources)
Paraphrased matches	7% (9 sentences)
Top source 1	2024 project report 'Smart Parking' - 6%
Top source 2	en.wikipedia.org/wiki/YOLO - 3%
Top source 3	Reference book: Deep Learning with Python - 2%
AI indicator by section	Ch.1 Medium (0.64), Ch.2-9 Low

Highlighted: 'YOLO frames object detection as a regression problem...' - paraphrase of source 2 (0.91)

The AI indicator is not proof; discuss with the student before any action.

SUBMISSIONS

https://integritycheck.app/

IntegrityCheck

UploadReportCorpusSubmissionsSettings

Submissions - III B.Sc CS (2026)

Batch: 2026Sort: Similarity

Reg. No	Title	Similarity	AI indicator	Status
21UCS1012	Fake news detector	41%	Medium	Revise
21UCS1045	Smart parking ANPR	18%	Medium (Ch.1)	Reviewed
21UCS1001	Crop disease detection	9%	Low	Accepted
21UCS1033	Flood warning system	6%	Low	Accepted

CORPUS DASHBOARD

https://integritycheck.app/

IntegrityCheck

UploadReportCorpusSubmissionsSettings

Institutional corpus

Documents

2,412

Fingerprints

18.6 M

Sentence vectors

1.24 M

Checks this month

386

Average similarity % by year

Year	Similarity %
2021	24
2022	20
2023	18
2024	16
2025	14
2026	12

Corpus composition

Type	Count
Previous project reports	1,812
Reference books / notes	426
Papers	174

CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The Plagiarism and AI-Generated Content Detector combines fingerprint-based exact matching, embedding-based paraphrase detection and a perplexity-based AI-text indicator to provide comprehensive integrity checks for academic reports. It achieved an F1-score of 0.97 for exact copying and 0.86 for paraphrase detection, and produces explainable highlighted reports with sources.

By including previous years' local reports in the corpus, the tool catches reuse that web-only tools miss. The AI indicator is presented with appropriate caution, supporting fair, discussion-based handling of suspected cases. The system offers colleges an affordable way to promote academic honesty.

8.2 Future Enhancement

- Cross-language plagiarism detection (Tamil-English translation).
- Source code plagiarism detection for programming assignments.
- Integration with LMS such as Moodle and Google Classroom.
- Better AI-text detection using newer zero-shot methods and calibration.
- Citation suggestion to help students cite sources properly.

CHAPTER IX

BIBLIOGRAPHY

1. Broder, A. Z., "On the resemblance and containment of documents", Proc. Compression and Complexity of Sequences, IEEE, 1997.
2. Schleimer, S., Wilkerson, D. S., and Aiken, A., "Winnowing: Local Algorithms for Document Fingerprinting", Proc. ACM SIGMOD, 2003.
3. Reimers, N., and Gurevych, I., "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", Proc. EMNLP-IJCNLP, 2019.
4. Gehrmann, S., Strobel, H., and Rush, A. M., "GLTR: Statistical Detection and Visualization of Generated Text", Proc. ACL System Demonstrations, 2019.
5. Mitchell, E., Lee, Y., Khazatsky, A., Manning, C. D., and Finn, C., "DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature", Proc. ICML, 2023.
6. Liang, W., Yuksekgonul, M., Mao, Y., Wu, E., and Zou, J., "GPT detectors are biased against non-native English writers", Patterns, Vol. 4, No. 7, 2023.
7. University Grants Commission, "Promotion of Academic Integrity and Prevention of Plagiarism in Higher Educational Institutions Regulations", 2018.

WEBSITE REFERENCE

1. <https://www.elastic.co/guide>
2. <https://www.sbert.net>
3. <https://huggingface.co/gpt2>
4. <https://www.ugc.gov.in>